海外广告智能落地页系统产品方案

本架构方案采用**Schema-First**的标准化设计，所有模块、接口、组件都有明确的结构化定义，AI 可以直接根据该方案自动生成全部的前后端代码，无需人工编写核心逻辑，实现 AI 直接完成开发。

---

1. 架构总览

整体采用标准化的分层架构，所有层的定义都结构化，AI 可以直接解析：

代码块

```
┌──────────────────────────────────────────────────────┐
│ 前端层：Next.js + React + Tailwind      │  -> 标准化组件+TypeScript Interface，AI直接生成
├──────────────────────────────────────────────────────┤
│ API层：Python + FastAPI + Pydantic      │  -> 标准化Schema+OpenAPI，AI直接生成接口
├──────────────────────────────────────────────────────┤
│ 数据层：SQLAlchemy + MySQL + Redis      │  -> 标准化模型定义，AI直接生成ORM
├──────────────────────────────────────────────────────┤
│ 分发层：Cloudflare R2 + CDN             │  -> 标准化配置，AI直接生成部署脚本
└──────────────────────────────────────────────────────┘
```

---

2. 标准化模块定义（AI 可直接解析）

所有模块都有明确的输入输出定义，AI 可以直接根据定义生成代码：

2.1 前端组件标准化定义

所有落地页组件都有明确的 TypeScript Interface 定义，AI 可以直接生成对应的 React+Tailwind 组件：

2.1.1 Hero 组件

代码块

```
/**
 * 头部Hero组件
 * AI可直接根据该Interface生成组件代码
 */
```

```
interface HeroProps {
  /** 主标题 */
  title: string;
  /** 副标题 */
  subtitle: string;
  /** 按钮文字 */
  buttonText: string;
  /** 按钮跳转链接 */
  buttonLink: string;
  /** 背景/主图URL */
  imageUrl: string;
  /** 是否是移动端 */
  isMobile: boolean;
}
```

2.1.2 卖点组件

代码块

```
interface FeatureProps {
  /** 卖点列表 */
  features: Array<{
    icon: string;
    title: string;
    description: string;
  }>;
  isMobile: boolean;
}
```

2.1.3 评价组件

代码块

```
interface TestimonialProps {
  /** 评价列表 */
  testimonials: Array<{
    name: string;
    role: string;
    avatar: string;
    content: string;
  }>;
  isMobile: boolean;
}
```

2.1.4 表单组件

代码块

```
interface FormProps {
  /** 页面ID，用于提交线索 */
  pageId: number;
  /** 表单字段 */
  fields: Array<{
    name: string;
    type: 'text' | 'email' | 'phone';
    label: string;
    required: boolean;
  }>;
  /** 提交按钮文字 */
  submitText: string;
}
```

2.1.5 CTA 组件

代码块

```
interface CtaProps {
  title: string;
  subtitle: string;
  buttonText: string;
  buttonLink: string;
  isMobile: boolean;
}
```

2.2 后端 API 标准化定义

所有 API 都有明确的 Pydantic Schema 定义，AI 可以直接生成 FastAPI 的接口代码：

2.2.1 认证接口

代码块

```
# 请求Schema
class LoginRequest(BaseModel):
    email: str
    password: str

# 响应Schema
class LoginResponse(BaseModel):
```

```
    access_token: str
    token_type: str
    user: UserSchema
```

2.2.2 物料接口

代码块

```
# 响应Schema
class AssetSchema(BaseModel):
    id: int
    team_id: int
    name: str
    type: str
    original_url: str
    webp_url: Optional[str]
    sizes: Dict[str, str]
    tags: Optional[List[str]]
    created_at: datetime
```

2.2.3 页面接口

代码块

```
# 请求Schema
class PageCreateRequest(BaseModel):
    name: str
    template_id: Optional[int]
    config: Dict[str, Any]
    tracking_config: Optional[Dict[str, str]]
    custom_domain: Optional[str]

# 响应Schema
class PageSchema(BaseModel):
    id: int
    team_id: int
    name: str
    slug: str
    custom_domain: Optional[str]
    config: Dict[str, Any]
    tracking_config: Optional[Dict[str, str]]
    is_published: bool
    created_at: datetime
    updated_at: datetime
```

2.2.4 表单提交接口

代码块

```
# 请求Schema
class LeadSubmitRequest(BaseModel):
    name: Optional[str]
    email: Optional[str]
    phone: Optional[str]
    message: Optional[str]

# 响应Schema
class LeadSubmitResponse(BaseModel):
    success: bool
    message: str
```

2.2.5 统计接口

代码块

```
# 事件上报请求
class EventRequest(BaseModel):
    event_type: str
    device_type: str
    user_agent: Optional[str]

# 统计响应
class PageStatsResponse(BaseModel):
    page_id: int
    visits: int
    clicks: int
    conversions: int
    leads: int
    mobile_visits: int
    desktop_visits: int
    mobile_conversion_rate: float
    desktop_conversion_rate: float
```

2.3 数据库模型标准化定义

所有表都有明确的字段定义，AI 可以直接生成 SQLAlchemy 的 ORM 模型：

代码块

```
# 团队模型
class Team(Base):
```

```python
    __tablename__ = "teams"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    notify_email: Mapped[Optional[str]]
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

# 用户模型
class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    team_id: Mapped[int] = mapped_column(ForeignKey("teams.id"))
    email: Mapped[str] = mapped_column(unique=True)
    password_hash: Mapped[str]
    role: Mapped[str]
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

# 物料模型
class Asset(Base):
    __tablename__ = "assets"
    id: Mapped[int] = mapped_column(primary_key=True)
    team_id: Mapped[int] = mapped_column(ForeignKey("teams.id"))
    name: Mapped[str]
    type: Mapped[str]
    original_url: Mapped[str]
    webp_url: Mapped[Optional[str]]
    sizes: Mapped[Dict[str, str]]
    tags: Mapped[Optional[List[str]]]
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

# 页面模型
class Page(Base):
    __tablename__ = "pages"
    id: Mapped[int] = mapped_column(primary_key=True)
    team_id: Mapped[int] = mapped_column(ForeignKey("teams.id"))
    name: Mapped[str]
    slug: Mapped[str] = mapped_column(unique=True)
    custom_domain: Mapped[Optional[str]]
    config: Mapped[Dict[str, Any]]
    tracking_config: Mapped[Optional[Dict[str, str]]]
    is_published: Mapped[bool] = mapped_column(default=False)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
    updated_at: Mapped[datetime] = mapped_column(default=datetime.utcnow, onupdate=datetime.utcnow)

# 线索模型
class Lead(Base):
    __tablename__ = "leads"
```

```python
    id: Mapped[int] = mapped_column(primary_key=True)
    page_id: Mapped[int] = mapped_column(ForeignKey("pages.id"))
    name: Mapped[Optional[str]]
    email: Mapped[Optional[str]]
    phone: Mapped[Optional[str]]
    message: Mapped[Optional[str]]
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

# 事件模型
class Event(Base):
    __tablename__ = "events"
    id: Mapped[int] = mapped_column(primary_key=True)
    page_id: Mapped[int] = mapped_column(ForeignKey("pages.id"))
    event_type: Mapped[str]
    device_type: Mapped[str]
    ip: Mapped[Optional[str]]
    user_agent: Mapped[Optional[str]]
    timestamp: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

---

3. AI 开发适配设计

本架构完全适配 AI 直接开发，原因如下：

1. **全结构化定义**：所有的组件、接口、模型都有明确的 TypeScript/Pydantic Schema，没有模糊的自然语言描述，AI 可以直接解析这些结构化的定义
2. **标准化技术栈**：采用的 Python+FastAPI+React+Tailwind 都是 AI 最熟悉的技术栈，AI 已经有大量的相关代码训练数据，可以直接生成符合规范的代码
3. **组件化设计**：所有的功能都拆分为小的、独立的组件，AI 可以逐个生成组件，不需要处理复杂的整体逻辑
4. **Schema-First**：所有的接口都遵循 OpenAPI 规范，AI 可以直接根据 OpenAPI 的定义生成前后端的对接代码，不需要人工对接

---

4. AI 直接开发的流程

只需要将本架构方案提供给 AI，AI 就可以自动完成全部的开发，流程如下：

1. **生成数据库层**：AI 根据数据库模型的定义，自动生成 SQLAlchemy 的 ORM 模型，以及数据库的初始化脚本
2. **生成后端 API**：AI 根据 API 的 Schema 定义，自动生成 FastAPI 的接口代码，包括参数校验、错误处理、权限校验

3. **生成前端组件**：AI 根据前端组件的 Interface 定义，自动生成 React+Tailwind 的组件代码，包括响应式适配
4. **生成前端页面**：AI 自动生成管理后台的页面，以及落地页的渲染逻辑
5. **生成部署配置**：AI 自动生成 Dockerfile、docker-compose.yml，以及 CDN 的配置脚本
6. **自动测试**：AI 自动生成基础的单元测试，确保代码的正确性

整个过程，不需要人工编写任何代码，AI 可以在 1 小时内完成全部 MVP 的代码生成，直接部署即可使用。

---

5. AI 生成代码示例

以下是 AI 根据本方案的定义，自动生成的代码示例：

5.1 Hero 组件代码（AI 生成）

代码块

```
import React from 'react';
import { HeroProps } from '@/types/components';

const Hero: React.FC<HeroProps> = ({
  title,
  subtitle,
  buttonText,
  buttonLink,
  imageUrl,
  isMobile
}) => {
  return (
    <section className={`
      relative overflow-hidden
      ${isMobile ? 'py-12 px-4' : 'py-20 px-8'}
    `}>
      <div className={`max-w-7xl mx-auto flex ${isMobile ? 'flex-col' : 'flex-row items-center'}`}>
        <div className={`${isMobile ? 'text-center mb-8' : 'w-1/2 pr-8'}`}>
          <h1 className={`font-bold text-gray-900 mb-4 ${isMobile ? 'text-3xl' : 'text-5xl'}`}>
            {title}
          </h1>
          <p className={`text-gray-600 mb-6 ${isMobile ? 'text-base' : 'text-xl'}`}>
            {subtitle}
          </p>
```

```
25            <a
26              href={buttonLink}
27              className="inline-block px-8 py-3 bg-blue-600 text-white font-
    medium rounded-lg hover:bg-blue-700 transition-colors"
28            >
29              {buttonText}
30            </a>
31          </div>
32          <div className={`${isMobile ? 'w-full' : 'w-1/2'}`}>
33            <img
34              src={imageUrl}
35              alt="Hero"
36              className={`rounded-lg shadow-lg w-full ${isMobile ? 'h-auto' : 'h-
    auto'}`}
37            />
38          </div>
39        </div>
40      </section>
41    );
42  };
43
44  export default Hero;
```

5.2 登录接口代码（AI 生成）

代码块

```
1   from fastapi import APIRouter, Depends, HTTPException, status
2   from fastapi.security import OAuth2PasswordRequestForm
3   from jose import JWTError, jwt
4   from datetime import datetime, timedelta
5   from pydantic import BaseModel
6   from typing import Optional
7   from sqlalchemy.ext.asyncio import AsyncSession
8
9   from app.db import get_db
10  from app.models import User
11  from app.utils import verify_password
12
13  # 配置
14  SECRET_KEY = "your-secret-key"
15  ALGORITHM = "HS256"
16  ACCESS_TOKEN_EXPIRE_MINUTES = 30
17
18  router = APIRouter(prefix="/api/v1/auth", tags=["auth"])
19
```

```python
class LoginRequest(BaseModel):
    email: str
    password: str

class LoginResponse(BaseModel):
    access_token: str
    token_type: str
    user: dict

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(minutes=15)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

@router.post("/login", response_model=LoginResponse)
async def login(
    request: LoginRequest,
    db: AsyncSession = Depends(get_db)
):
    # 查找用户
    user = await db.get(User, request.email)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect email or password",
        )
    # 验证密码
    if not verify_password(request.password, user.password_hash):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect email or password",
        )
    # 生成token
    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.email}, expires_delta=access_token_expires
    )
    # 返回
    return {
        "access_token": access_token,
        "token_type": "bearer",
        "user": {
```

```
            "id": user.id,
            "email": user.email,
            "role": user.role,
            "team_id": user.team_id
        }
    }
```

---

6. 部署与上线

AI 生成完所有代码后，只需要执行以下命令即可一键部署：

代码块

```
# 启动所有服务
docker-compose up -d

# 初始化数据库
docker-compose exec backend alembic upgrade head
```

部署完成后，系统就可以直接使用了。

（注：文档部分内容可能由 AI 生成）